Plateforme de gestion des activités associatives étudiantes

Stack technique :

Frontend : React

Backend : Node.js (Express)

Base de données : MySQL

Authentification : JWT

Notifications : Email via SMTP

Cahier des charges :

1. Présentation Générale

1.1 Nom du projet

« CampusConnect »

1.2 Contexte

Les associations étudiantes organisent régulièrement des événements (conférences, compétitions, formations, soirées, etc.).

Cependant :

- Il n'existe pas de système centralisé.
- Les inscriptions sont mal gérées.
- Les communications sont dispersées.
- Il n'y a pas de statistiques fiables.

1.3 Problématique

Comment centraliser la gestion des événements associatifs universitaires et améliorer la communication, la participation et le suivi administratif ?

1.4 Objectifs

Objectif principal

Créer une plateforme web permettant :

- La gestion des événements
- L'inscription des étudiants
- Le suivi des participations
- L'envoi de notifications automatiques

Objectifs secondaires

- Améliorer la visibilité des activités
- Simplifier la gestion administrative
- Fournir des statistiques d'engagement

2. Périmètre du Projet

2.1 Inclus dans le projet

- Authentification sécurisée (JWT)
- Gestion des rôles
- CRUD événements
- Inscription aux événements
- Notifications email
- Tableau de bord
- Statistiques simples

3. Parties Prenantes

- Étudiants
- Responsables d'associations
- Administration universitaire
- Équipe projet (PO + SM + Dev)

4. Description Fonctionnelle

4.1 Gestion des utilisateurs

Rôles

1. Étudiant
2. Responsable d'association
3. Administrateur

4.1.1 Inscription

Un utilisateur peut :

- Créer un compte
- Choisir son rôle (validation admin si responsable)
- Recevoir un email de confirmation

4.1.2 Connexion

- Authentification par email + mot de passe
- Génération d'un JWT
- Token stocké côté client

4.2 Gestion des associations

Un administrateur peut :

- Créer une association
- Supprimer une association
- Valider un responsable

4.3 Gestion des événements

Un responsable peut :

- Créer un événement
- Modifier un événement
- Supprimer un événement
- Définir :
    - Date
    - Lieu
    - Description
    - Nombre maximal de participants

4.4 Inscription aux événements

Un étudiant peut :

- Voir liste des événements
- Voir détails d'un événement
- S'inscrire si places disponibles
- Se désinscrire

4.5 Notifications Email (SMTP)

Emails envoyés automatiquement :

- Confirmation inscription
- Confirmation participation à un événement
- Rappel avant événement (optionnel)
- Annulation événement

4.6 Tableau de Bord

Responsable

- Nombre d'inscrits
- Taux de participation
- Historique événements

Administrateur

- Nombre total événements
- Associations actives
- Nombre utilisateurs

5. Exigences Non Fonctionnelles

5.1 Sécurité

- Hash mot de passe (bcrypt)
- Authentification JWT
- Middleware d'autorisation par rôle
- Validation des données côté backend

5.2 Performance

- Temps de réponse < 10 secondes
- Pagination pour listes

5.3 Compatibilité

- Responsive (mobile + desktop)
- Navigateurs modernes

5.4 Maintenabilité

- Architecture MVC
- Code structuré
- Séparation frontend/backend

6. Architecture Technique

6.1 Frontend (React)

Structure :

- Pages :
  - Login
  - Register
  - Dashboard
  - Liste événements
  - Détail événement
  - Admin panel
- Gestion état :
  - Context API ou Redux (optionnel)
- Appels API :
  - Axios

6.2 Backend (Node.js + Express)

Architecture :

- controllers/
- routes/
- models/
- middlewares/
- config/

Fonctionnalités :

- REST API
- Middleware auth JWT
- Middleware rôle

6.3 Base de Données (MySQL)

Tables principales

users

- id
- name
- email
- password

- role
- created_at

associations

- id
- name
- description
- created_at

events

- id
- association_id
- title
- description
- date
- location
- max_participants
- created_at

registrations

- id
- user_id
- event_id
- created_at

6.4 Notifications SMTP

- Nodemailer
- Configuration serveur SMTP
- Template email HTML

7. Contraintes Techniques

Utilisation de :

- React
- Node.js (Express)
- MySQL
- JWT
- SMTP

- Code versionné sur GitHub
- Déploiement Docker (optionnel)

8. Planning Prévisionnel (Scrum)

Sprint 0 :

- Setup projet
- Conception BDD
- Architecture
- Backlog

Sprint 1 :

- Authentification
- CRUD événements

Sprint 2 :

- Inscription événements
- Dashboard

Sprint 3 :

- Notifications
- Tests
- Optimisation

9. Livrables

- Code source
- Documentation API
- Diagramme UML
- Diagramme BDD
- Backlog produit
- Rapport projet
- Démo fonctionnelle

10. Risques Identifiés

- Problèmes sécurité JWT
- Mauvaise gestion rôles
- Envoi email bloqué (SMTP configuration)
- Mauvaise gestion concurrence (inscriptions simultanées)